

WebTransport

Page No.

Date: / /

→ New technology.

→ websocket vs webTransport (Replacement?)

* Intro

→ webTransport is a new web communication protocol built on HTTP/3 (QUIC)

• Benefits

→ Prevent (Head of line blocking issues)

→ TCP based had to receive the packet if lost, resulting in multiple request check.

→ Build on top of HTTP/3 QUIC

→ Maintains bidirectional messaging like websockets.

→ Supports both TCP/UDP

→ Browser support is increasing

* WebSocket

→ Bidirectional communication (improvement over polling and server sent events)

→ Lower overhead (no HTTP headers after handshake)

→ Real-time (TCP)

→ Persistence communication

✓✓ Websockets Limitations

→ Head of Line Problems

→ Single stream per connection

• If you're sending different types of data, large message can clog the pipe for other data types.

→ Reliability constraints

• Always ensures reliable, ordered delivery
Inefficient for live video or game state.

Request with TCP + TLS (HTTP/2)

→ Three way Handshake

→ TLS (creates) a separate Tunnel Secure

→ Upgrade to websocket

→ Websocket conn, established.

• Uses single Bidirectional Message Channel

Request with QUIC

→ Three way QUIC Handshake (includes TLS)

→ Upgrade to webtransport

→ webtransport connection established.

• Less Round Trips

• UDP

• Uses same UDP conn for both types of connection, streams and datagrams.

we can have both reliable and non-reliable datachannel within single UDP connection.

Web Transports

- Secured with (TLS)
- Faster handshake
- Supports both Streams and Datagrams within a single connection.
- Multiplexed Streams (uses different channel for different data types)

* Connection Migration

- Users / device might move from one type of connection to another. For example, WiFi to LTE (Telecom)

↳ If the connection was established with WebSocket, then on network changed, the connection has to be reestablished from the beginning (Different IP address, as network changes)

↳ But, WebTransport (QUIC) uses WebTransport connection ID instead of IP addresses, so when we switch network and get a new IP address the WebTransport automatically reconnects to the server using the same connection ID.

* Real world Examples

- High Frequency Trading Platforms
 - Market data via UDP (latest price matters)
 - Order confirmations (TCP reliable stream)
 - Banking (reliable stream)
 - WebTransport wins, over websocket
- Cloud Gaming
 - Video transfer [UDP]
 - Player input [UDP]
 - WebTransport wins
- Real-time collaborative Design Tools
 - cursor movements (UDP, for real time)
 - Document changes (TCP, every change should be preserved)
 - Voice chat (UDP)
 - WebTransport wins:

* when to use which

- Websocket
 - Simple real-time messaging (chat, noti)
 - Universal browser support required
 - Single data type
 - Reliable delivery essential
- WebRTC
 - Peer to peer connection
 - Real-time audio, video
 - No server bandwidth costs (P2P file sharing)

Big Big Big
Big Big

Page No.

Date : / /

→

- WebTransport
 - High performance client-server
 - tailored reliability requirements
 - Multiple independent data streams
 - Ultra low latency applications
 -

How Implementation works in real world.

In real world application, we should not rely on single Protocol.

So, the basic principle is:

- Use webtransport
 - Safari no support
 - Corporate network blocks QUIC (Firewall, Packet Inspect)

↳ If these, then
- Fall Back to websockets
 - Old browsers don't support
 - Corporate network can block WS (Firewall, Packet Inspect)

↳ If these, then
- Fall back to server sent Events.

So, in real world applications.

Gracefull fallback strategy is used.